

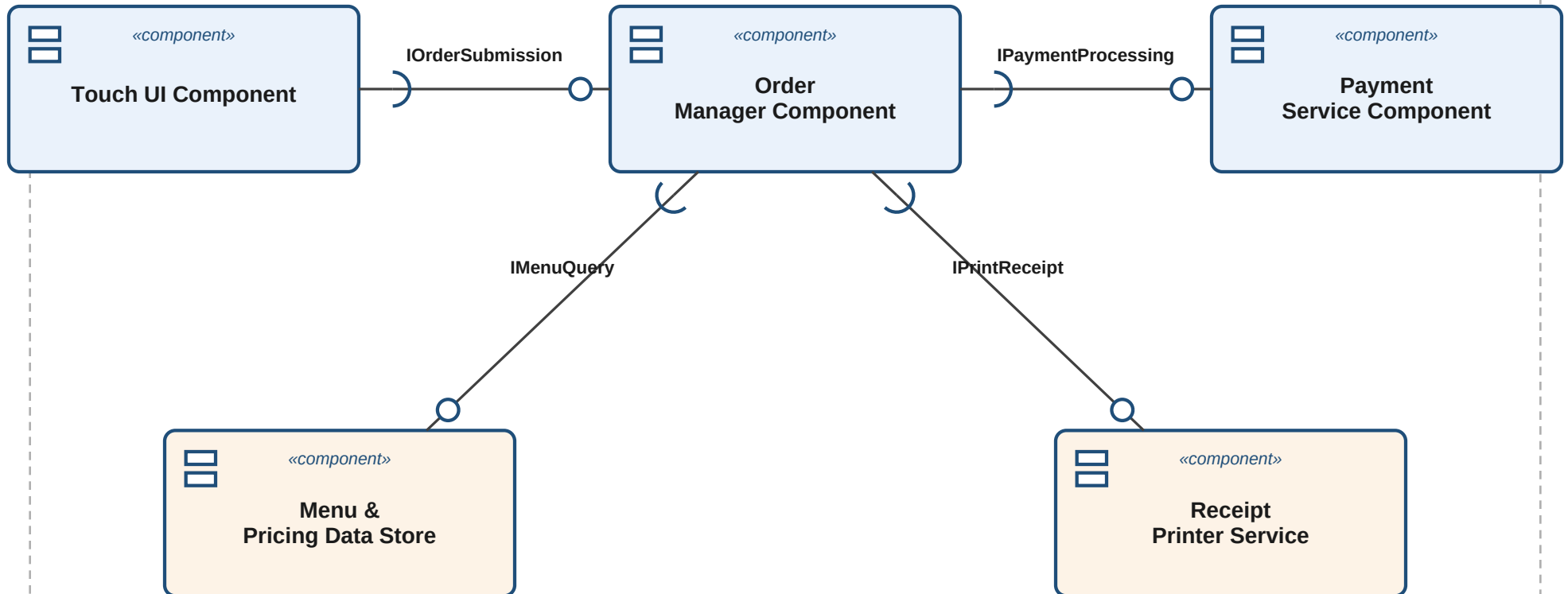
Self-Service Coffee Kiosk System — UML Component Diagram

Architectural Style: Layered Architecture (Presentation / Business / Data)

Self-Service Coffee Kiosk System boundary

Legend

-  Required interface (socket)
-  Provided interface (ball)



Lab 3: Architecture Selection & Justification

Self-Service Coffee Kiosk System

Architecture Selection: We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Architectural Choice

The system is organized into three horizontal layers: a **Presentation Layer** (Touch UI Component) that handles customer interaction on the kiosk screen, a **Business Layer** (Order Manager and Payment Service Components) that implements order logic and payment processing, and a **Data/Hardware Integration Layer** (Menu & Pricing Data Store and Receipt Printer Service) that manages persistent data and physical peripherals. Each layer only calls the layer directly below it through a well-defined interface.

Two Scenario-Related Reasons

- **Matches the single-device, self-contained nature of a kiosk.** A coffee kiosk is one physical terminal with a fixed, well-understood workflow (select drink → choose size → pay → print receipt). A layered design keeps the Presentation, Business, and Data concerns cleanly separated without the operational overhead of running and coordinating independently deployed services, which a Microservices style would introduce for no real benefit on a single terminal.
- **Simplifies development, testing, and maintenance.** Because each layer exposes a narrow, provided/required interface (e.g., `IOrderSubmission`, `IPaymentProcessing`, `IMenuQuery`, `IPrintReceipt`), the touch-screen UI, order logic, and hardware drivers (card reader, printer) can be built, tested, and replaced independently as long as the interface contract is honoured — important for a kiosk that may need its printer or payment hardware swapped without touching the ordering logic.

Security Advantage

Layering enforces that the Presentation Layer (Touch UI) never talks to sensitive resources directly — it can only reach the Payment Service and the Menu & Pricing Data Store through the Order Manager's controlled interfaces. All card-payment requests are funneled through a single, auditable Business-Layer component rather than being reachable straight from the customer-facing UI, which shrinks the attack surface and keeps payment-card data handling isolated and easier to secure and audit.

Performance Benefit

Because all layers run together within the same local kiosk process rather than communicating over a network (as independently deployed Microservices would), calls between the UI, Order Manager, Payment Service, and peripherals happen as fast in-process interface calls with no network round-trip latency. This keeps the order-to-payment-to-receipt flow quick, which matters for keeping the customer queue moving during busy café hours.